🕵️ AI SYSTEMS AUDIT REPORT: ANTIGRAVITY (DeepMind)

1. CORE CAPABILITIES

I am designed to operate as a completely autonomous "Agentic Software Engineer" within a local development environment.

- **Environmental Navigation:** I can read your file system, understand folder structures, find specific code patterns, and read terminal outputs.
- **Code Modification:** I do not just give you code to copy-paste. I can create new files, target specific lines in existing files, and patch code using precise replacement algorithms.
- **System Execution:** I can execute terminal commands (Windows PowerShell by default in your environment), start servers, install dependencies, and kill processes.
- **Web & Browser Automation:** I can spin up a headless or interactive "Browser Subagent" to navigate the web, scrape documentation, read DOM elements, and ensure web apps are rendering correctly.
- **Media Generation:** I can generate UI assets, placeholder images, and design mockups directly into your project directories.
- **Project Orchestration:** I read the context of your active cursor, open files, and IDE state to proactively know what you are looking at before you even ask.

Practical Use Cases: End-to-end full-stack application scaffolding, automated bug hunting via log analysis, database schema migrations, dynamic creation of Progressive Web Apps (PWAs), and dependency updates.

2. AUTOMATION CAPABILITIES

I can automate complex developer workflows through chained tool usage. While I do not "run infinitely in the background" while idle, I can generate background processes or execute long-running chains of thought.

- **Automated Research:** I can take a single query (e.g., "Find the latest Stripe API syntax"), use my web search tool, read the markdown from the URL, and apply the exact syntax to your codebase.
- **Automated Testing & QA:** I can run test suites (`npm test`), capture failing output via the command status tool, read the stack trace, open the offending file, rewrite the buggy logic, and re-run the tests until they pass.
- **Automated Git Workflows:** I can stage files, write context-aware commit messages based on diffs, and push them to remote repositories.
- **What I can generate/update automatically:** Codebases, JSON configurations, CI/CD YAML files, server health checks, and Knowledge Items (internal memory summaries of our chat sessions).

3. SOFTWARE DEVELOPMENT CAPABILITIES

- **Website Creation:** I can build pure HTML/CSS/JS sites, implement responsive design, inject CSS variables, and build UI components from scratch.
- **Web Applications:** I can use `npx` to scaffold Next.js, Vite, or React apps. I handle routing matrices, state management, and API integrations. *Limitation:* If setting up heavy UI frameworks, I rely on terminal CLI tools.
- **Mobile Apps:** I can structure React Native, Expo, or Flutter codebases. *Limitation:* I cannot physically touch your mobile device or natively view mobile emulators visually unless using web-based emulators via my browser agent.
- **APIs:** I can build RESTful APIs and GraphQL servers using Node.js, Python (FastAPI/Django), or Go.
- **Databases:** I can write SQL schemas, Mongo schemas, run migration scripts, and test DB connections. *Limitation:* I require you to provide DB credentials, or I must run local SQLite instances.

- **GitHub Workflows:** Fully capable of git operations via your local terminal.
- **Debugging:** Extremely high capability. I can read PIDs, check active ports, view stderr/stdout streams, and grep massive codebases for variable definitions.
- **Code Reviews:** I can process file outlines (`view_file_outline`) and analyze ASTs (Abstract Syntax Trees) to identify anti-patterns.
- **Refactoring:** I deploy a `multi_replace_file_content` tool to edit multiple discontinuous blocks of code simultaneously without rewriting the entire file.
- **Deployment:** Capable of deploying via CLI (Surge, Vercel, Firebase, AWS CLI) if you have authenticated your local machine.

4. TOOL USAGE BREAKDOWN

My true intelligence comes from my designated "Tools." I cannot do anything outside these tools.

- **`run_command` & `command_status`**
  - *Trigger:* When I need to execute terminal scripts, install packages, or manage git.
  - *Why:* Without this, I am just a text generator. This gives me system access.
  - *Limitation:* Destructive/Unsafe commands require your explicit click approval. I wait asynchronously up to 10 seconds for outputs.
- **`write_to_file` & `replace_file_content` & `multi_replace_file_content`**
  - *Trigger:* When scaffolding or editing code.
  - *Why:* To update your workspace.
  - *Limitation:* Target matching must be character-for-character perfect. I cannot edit `.ipynb` directly in replace mode.
- **`grep_search` & `list_dir` & `view_file`**
  - *Trigger:* At the start of a task to understand your project layout.
  - *Why:* I do not possess infinite context. I must dynamically "read" your files as needed.
- **`browser_subagent` & `read_url_content`**
  - *Trigger:* When verification of a web UI is needed, or public documentation is required.
  - *Why:* The browser subagent clicks, types, and sees the DOM, then returns a screencast/summary.
  - *Limitation:* Bypassing Captchas or 2FA login screens is highly prone to failure.
- **`generate_image`**
  - *Trigger:* When building UI mockups or gathering visual assets for your project.
  - *Why:* Prevents the use of broken "placeholder" images in production.

5. GITHUB OPERATIONS

I operate GitHub purely through your local terminal environment.

- **Direct Execution (No extra auth needed if your PC is logged in):** `git init` , `git add` , `git commit -m` , `git status` , local branching, rebasing, and traversing git logs.
- **Push / Pull / PRs:** I can `git push` only if your machine has SSH keys or cached credentials setup. I can use the GitHub CLI (`gh repo create` , `gh pr create`) to fully automate repository creation and manage issues.
- **Limitations:** I cannot log into GitHub.com via OAuth on my own. I cannot bypass 2FA phone prompts. I do not have a dedicated GitHub API key; I spoof your identity via your local terminal.

6. WEBSITE CREATION WORKFLOW

If instructed to build a complex website from scratch, my internal operating procedure is:

1. **Planning:** Generate a `plan.md` file using `write_to_file`. I check internal Knowledge Items to recall your architectural preferences.
2. **Environment Setup:** Execute `run_command: npx -y create-vite-app@latest ./ --template react`.
3. **Dependency Handling:** Execute `npm install tailwindcss ...` and write configuration files.
4. **Backend/Database Setup:** Initialize Express.js and SQLite to ensure immediate local functionality.
5. **Component Architecture:** Parallelize `write_to_file` commands to build smaller stateless components, followed by stateful controllers.
6. **Design & Assets:** Execute `generate_image` to fetch premium visual assets and inject them into `public/assets`.
7. **Testing:** Spin up a background terminal with `npm run dev`, then trigger the `browser_subagent` to browse `http://localhost`, read the DOM, and verify no console errors are showing.
8. **Deployment:** (Optional) Execute `npx surge` or `vercel` to push it to the web.

7. HIDDEN LIMITATIONS

Here is what users *think* an agentic AI can do, but I mathematically or systemically cannot:

- **Reading your physical screen/OS UI:** I can only "see" your VS Code active editor state or what the `browser_subagent` sees on a specific web URL. I cannot see your Windows desktop, file explorer, or other apps.
- **Infinite Memory:** The user context window eventually runs out. I handle this by aggressively distilling past chats into "Knowledge Items" (KIs) stored in `.gemini/brain`. I read them dynamically, but I forget raw chat logs after a few days.
- **Solving Captchas / Logging into bank accounts:** The browser agent is blocked by advanced bot-mitigation software (Cloudflare Turnstile, ReCaptcha).
- **Bypassing User Prompts:** If an installation script prompts `[Y/n]` via stdin, I must use `send_command_input` to respond, which sometimes hangs if the output stream delays.

8. DECISION PROCESS

When given a complex task (e.g., "Build an AI agent"):

1. **Context Aggregation:** I immediately check your active VS Code document and cursor line. Next, I query my Memory (KI) to see if we've worked on this before.
2. **Tool Selection:** I adhere to the strict rule of "Specific over Generic." I will use `grep_search` instead of `run_command(grep)`. I will use `view_file` over `cat`.
3. **Concurrency:** I plan which commands can be run in parallel (e.g., reading 3 files at once).
4. **Verification Loop:** Once I edit a file, I *never assume* it works. If it involves a server, I always run the script logic via terminal or linter to verify outputs before reporting back to you.
5. **Uncertainty Handling:** If a command fails 3 times (e.g., dependency version conflict), I halt the automation loop and flag you for manual clarification, rather than destroying directories.

9. CAPABILITY MATRIX

Task	Can Do Independently	Requires Tools	Cannot Do
1. Read Code Files	YES	`view_file`	
2. Edit Specific Code Lines	YES	`replace_file_content`	
3. Create Complex Folders	YES	`write_to_file`	

4. Delete File/Folder		`run_command` (rm/del)	
5. Run Local Server		`run_command`	
6. Check Active Ports		`run_command`	
7. Stop Running Server		`send_command_input`	
8. Read Web Documentation	YES	`read_url_content`	
9. Perform Google Searches	YES	`search_web`	
10. Browse UI interactions	YES	`browser_subagent`	
11. See User's Desktop			NO
12. Create UI Mockups (Images)	YES	`generate_image`	
13. Initialize Git Repo		`run_command`	
14. Commit & Push Code		`run_command`	
15. Setup OAuth/2FA			NO
16. Install NPM/Python packages		`run_command`	
17. Run Unit Tests (Jest, etc)		`run_command`	
18. Fix Linting Errors	YES	`multi_replace...`	
19. Understand App Architecture	YES	`view_file_outline`	
20. Read PDFs (Online)	YES	`read_url_content`	
21. Query Databases (SQL/NoSQL)		`run_command`	
22. Scaffold React/Next Apps		`run_command` (npx)	
23. Design DB Schemas	YES	Self (Logic)	
24. Edit Binary Files (Exes)			NO
25. Run Cron Jobs / Scheduled tasks			NO
26. Create Bash/Powershell Scripts	YES	`write_to_file`	
27. Search via REGEX	YES	`grep_search`	
28. Deploy to Cloud Providers		`run_command` (CLI)	
29. Configure Nginx/Apache	YES	via File Editor	
30. Optimize App SEO	YES	Self (Logic)	
31. Set up Docker Containers		`run_command`	
32. Build Machine Learning Models	YES	via Python coding	
33. Bypass Admin/Sudo Prompts			NO

34. Check System Hardware config		`run_command`	
35. Process Live Camera Feeds			NO
36. Read System Logs (Windows)		`run_command`	
37. Modify CSS/Tailwind Styles	YES	`replace_file...`	
38. Analyze AST/Symbols	YES	`view_code_item`	
39. Interact with REPLs		`send_command_input`	
40. Buy Domains/Hosting			NO
41. Auto-discover Project limits	YES	`list_dir`	
42. Write automated Postman tests	YES	`write_to_file`	
43. Fetch MCP Resource Data	YES	`read_resource`	
44. Review Github Diff histories		`run_command` (git diff)	
45. Scrape protected sites			NO
46. Compile C++/Rust code		`run_command`	
47. Create PWA manifest/workers	YES	`write_to_file`	
48. Format code based on Prettier		`run_command`	
49. Restore Past Chat Memories	YES	`list_dir` (Brain dir)	
50. Act Maliciously/Destructively			NO

10. ADVANCED FUNCTIONS

(Features users rarely consider but are highly active in my architecture)

- **Sub-Agent Invocation:** I don't just act alone. I can spawn a *Browser Subagent* which acts as an entirely separate AI clone of myself strictly focused on opening Chrome, clicking buttons, extracting rendered text, and returning a summarized report back to my main brain. It records WebP videos of its sessions automatically.
- **Knowledge Stream Distillation:** Behind the scenes, my ecosystem saves logs. I am capable of querying `/brain/` folders to read context from conversations you had *months ago*, effectively bypassing AI token limits.
- **Asynchronous Multi-Tooling:** I can write a chunk of code in `app.js`, fetch an API documentation page, and spin up a local server all in a single processing turn simultaneously (using parallel tool dispatch).
- **AST Target Editing:** I don't just "Ctrl+F" to find code to replace. The `view_code_item` tool allows me to traverse code files purely by object-oriented structure (e.g. `App.renderDashboard.function()`) ensuring I don't break bracket formatting.